PROJECT: Secure File Sharing with S3 and CloudFront

Services: Amazon S3, Amazon CloudFront, AWS IAM, CloudFront Public Key, CloudFront Key Group, AWS CloudShell, OpenSSL

Project Goal: To securely share private files by storing them in Amazon S3 and delivering them through CloudFront using signed URLs so that only authorized users can access the content.

Skills: Secure file storage, CloudFront CDN configuration, signed URL implementation, IAM access control, and AWS security best practices.

Definition: This project involves designing and implementing a secure file-sharing solution using Amazon S3 and Amazon CloudFront. Files are stored privately in an S3 bucket with public access blocked, while CloudFront acts as a secure content delivery layer. Access to the files is controlled using CloudFront signed URLs and key groups, ensuring that only authorized users with valid, time-limited URLs can download or view the content. The solution improves security, enforces controlled access, and provides scalable and efficient content delivery.

Scope of the Project: The scope of this project includes securely storing private files in Amazon S3, configuring Amazon CloudFront as a content delivery network, and restricting file access using signed URLs and key groups. It covers setting up IAM permissions to enforce least-privilege access, blocking public access to S3 objects, and validating secure access through CloudFront. The project focuses on secure content distribution, access control, and scalability, and can be extended to support user authentication, automatic URL generation, logging, and expiration-based access for real-world file-sharing applications.

Methodologies:

Method 1: S3 Public Access with Object URLs

Files are stored in S3 and accessed directly using S3 object URLs by making the bucket or objects public.

Limitations:

- Files are publicly accessible
- No access control or expiration
- High security risk
- Not suitable for confidential data

Method 2: S3 Pre-Signed URLs

S3 generates temporary pre-signed URLs to allow limited-time access to private objects.

Limitations:

- URLs are generated per object and per request
- No CDN caching benefits
- Limited scalability for large user traffic
- Performance depends on S3 region

Method 3: EC2 + Application Server

An EC2 instance hosts an application that authenticates users and serves files from S3.

Limitations:

- High operational overhead
- Requires server management and scaling
- Increased cost
- Single point of failure if not designed properly

Method 4: API Gateway + Lambda

API Gateway triggers Lambda to validate users and generate temporary access to S3 objects.

Limitations:

- Added architectural complexity
- Cold start latency
- Higher cost for frequent requests
- Requires multiple integrations

Why the Chosen Method (S3 + CloudFront Signed URLs) Is Better

- The S3 and CloudFront signed URL approach provides the best balance of **security, scalability, performance, and cost**.
- Files remain completely private in S3 while CloudFront securely delivers them using time-limited signed URLs.
- This method eliminates the need for managing servers, reduces latency through global edge locations, and ensures that only authorized users can access the content. CloudFront key groups allow easy key rotation and centralized access control, making this approach more secure and scalable than direct S3 access or server-based solutions.

Services Used in this Project:

1. Amazon S3 (Simple Storage Service)
Definition: Amazon S3 is a scalable object storage service used to store and retrieve data securely.
Purpose: To store private files securely without allowing public access.
Role in the Project: S3 acts as the storage layer where all files are uploaded and kept private, and it serves as the origin for CloudFront to deliver files securely.

2. Amazon CloudFront
Definition: Amazon CloudFront is a Content Delivery Network (CDN) service that distributes content securely with low latency.
Purpose: To deliver stored files securely and efficiently to authorized users.
Role in the Project: CloudFront fetches files from the private S3 bucket and serves them only through signed URLs, ensuring secure and fast content delivery.

3. AWS IAM (Identity and Access Management)
Definition: AWS IAM is a service used to manage permissions and access to AWS resources.

Purpose: To control which services can access the S3 bucket and CloudFront resources.
Role in the Project: IAM policies ensure that only CloudFront has permission to access the private S3 bucket, enforcing least-privilege access.

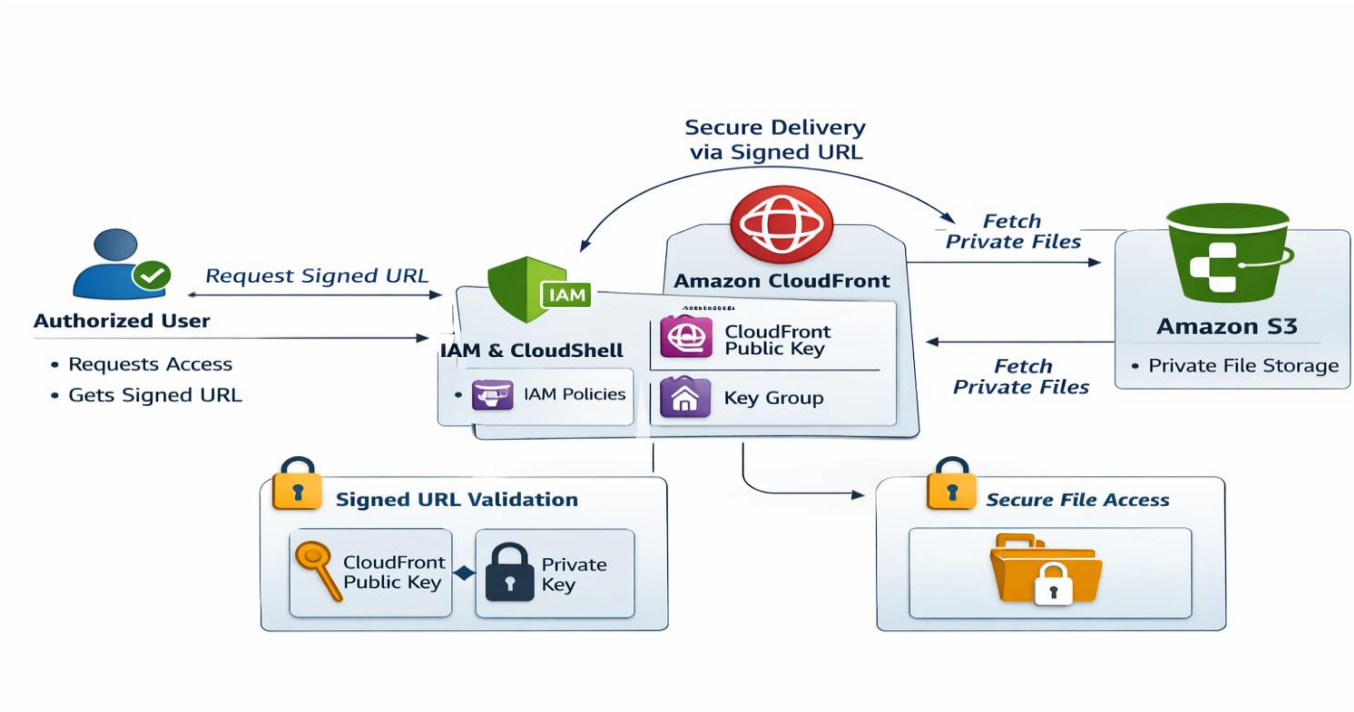
4. CloudFront Public Key
Definition: A CloudFront public key is used to verify the authenticity of signed URLs.
Purpose: To validate requests made using signed URLs.
Role in the Project: The public key verifies that the request was signed with the correct private key before granting access to the file.

5. CloudFront Key Group
Definition: A key group is a collection of public keys used by CloudFront for signed URL validation.
Purpose: To manage and associate public keys with CloudFront distributions.
Role in the Project: The key group is attached to the CloudFront distribution to authorize access using signed URLs.

6. AWS CloudShell
Definition: AWS CloudShell is a browser-based command-line environment.
Purpose: To generate keys and signed URLs securely without using a local machine.
Role in the Project: CloudShell was used to generate RSA key pairs and run Python scripts for signed URL creation.

7. OpenSSL
Definition: OpenSSL is a cryptographic toolkit for encryption and key generation.
Purpose: To generate secure RSA public and private keys.
Role in the Project: OpenSSL was used to create the cryptographic keys required for CloudFront signed URL authentication.

Architecture Diagram



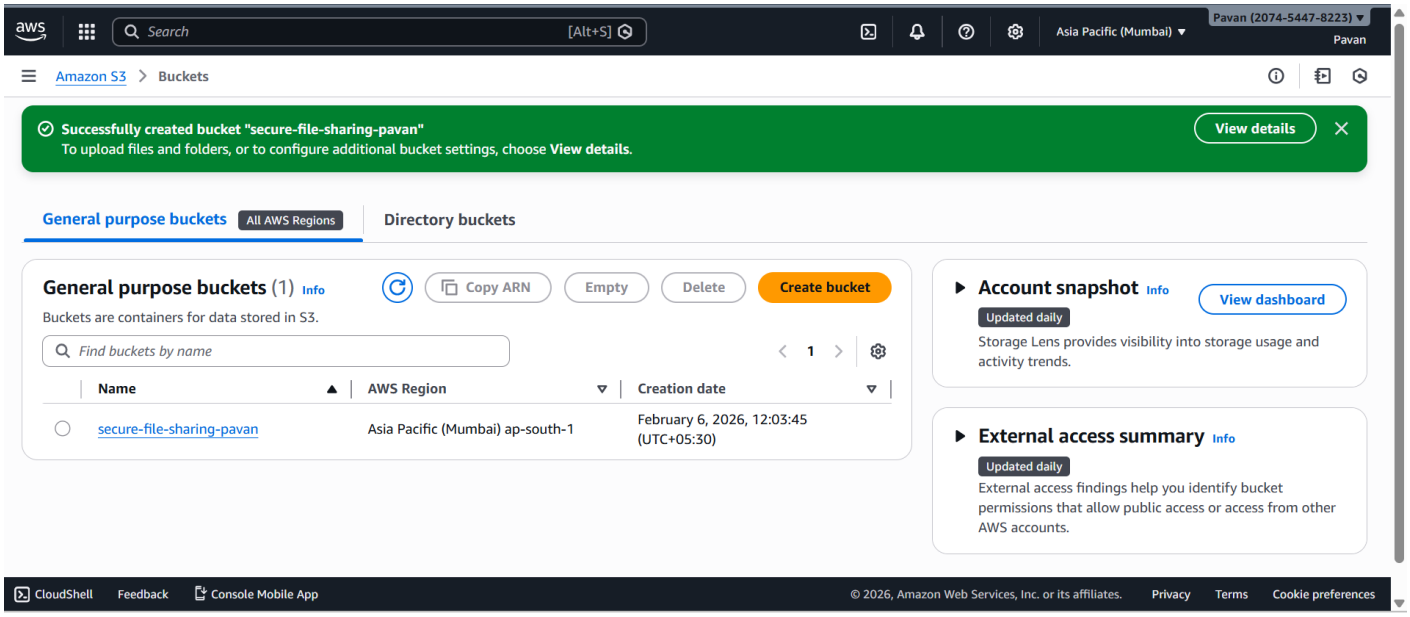
Implementation and Execution of a Secure File Sharing with S3 and CloudFront

Phase 1: Create a Secure S3 Bucket

1. Go to AWS Console → S3 → Create bucket
   - Bucket name: `secure-file-sharing-pavan`
   - Region: Any (same region as CloudFront origin recommended)

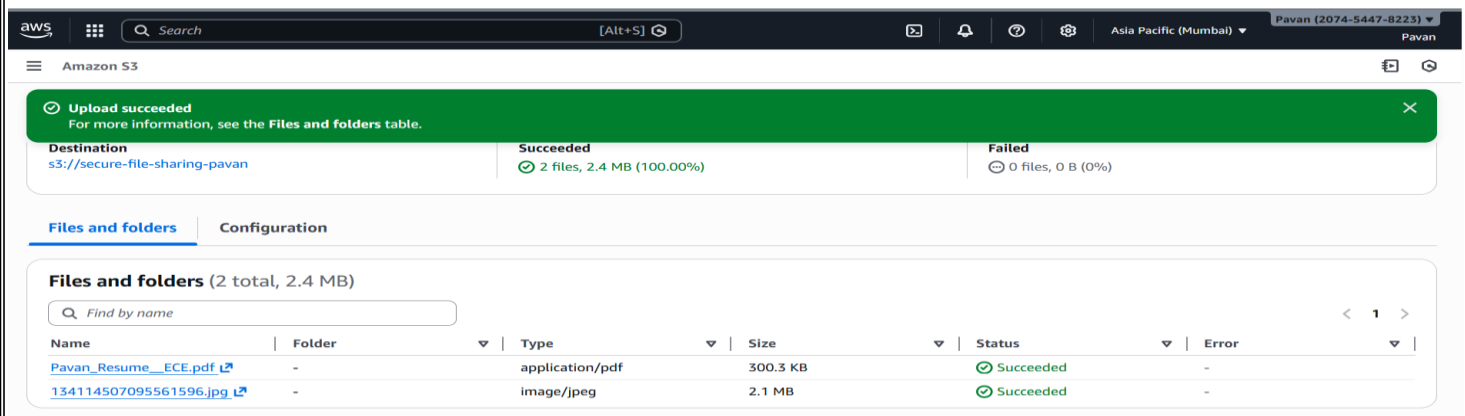
2. Block Public Access
   - ✓ Enable **Block all public access**

3. Create Bucket
   - Click **Create bucket**



Phase 2: Upload Files to S3

1. Open the bucket
2. Click **Upload**
3. Upload sample files (PDF, image, video)
4. Keep permissions default (private)

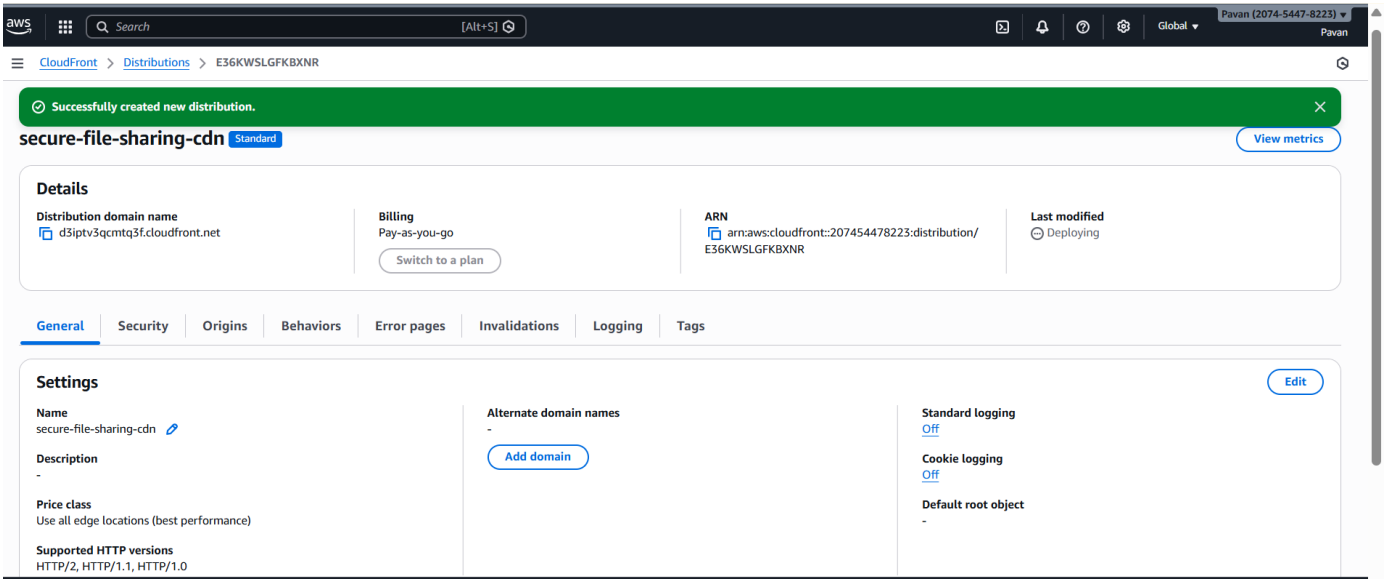


Phase 3: Create CloudFront Distribution

1. Go to CloudFront → Create distribution
2. Origin Settings
    - Origin domain: **Select your S3 bucket**
    - Origin access: **Origin Access Control (OAC)**
    - Create new OAC
    - Signing behavior: **Sign requests**

3. Default Cache Behavior
    - Viewer protocol policy: **Redirect HTTP to HTTPS**
    - Allowed HTTP methods: **GET, HEAD**
    - Restrict viewer access: **Yes (Use Signed URLs)**

4. Create Distribution
    - Wait until status becomes **Enabled**



Phase 4: Attach S3 Bucket Policy for CloudFront

1. Go to S3 → Bucket → Permissions → Bucket Policy

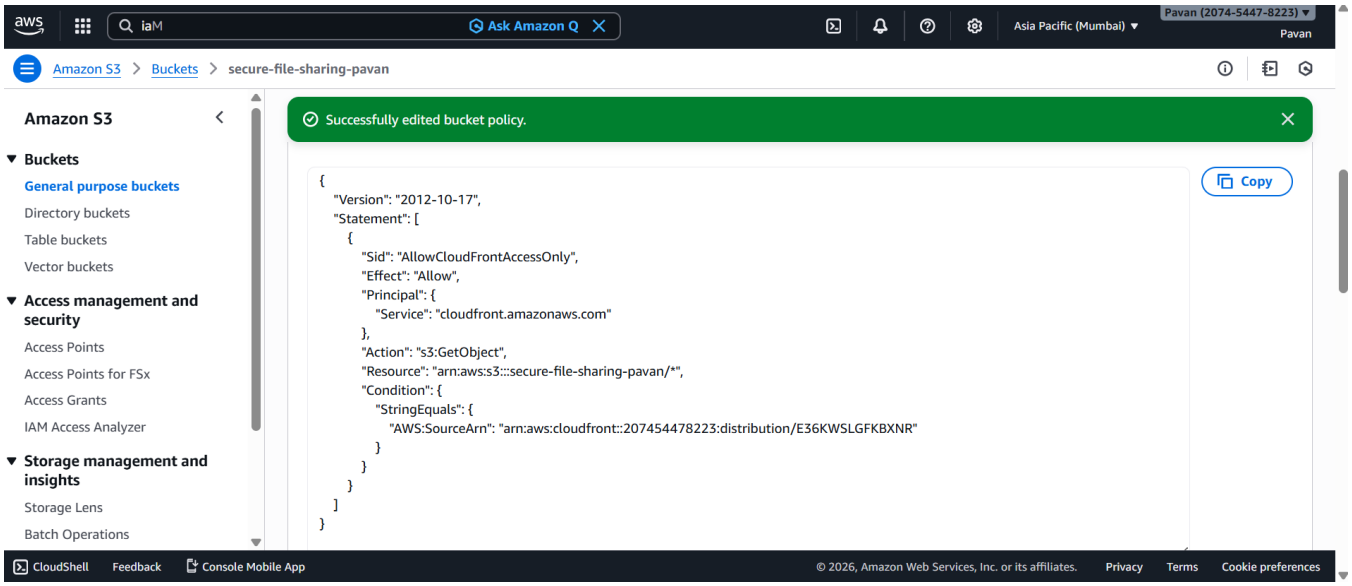
Add this policy (replace values):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::secure-file-sharing-pavan/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<DISTRIBUTION_ID>"
        }
      }
    }
  ]
}
```

2. Save Policy

Now:

- S3 objects can only be accessed via CloudFront
- Direct S3 URL access is blocked



Phase 5: CREATE A PUBLIC / PRIVATE KEY PAIR (LOCAL)

- On your laptop (Windows)
- Open **Command Prompt / PowerShell** and run:

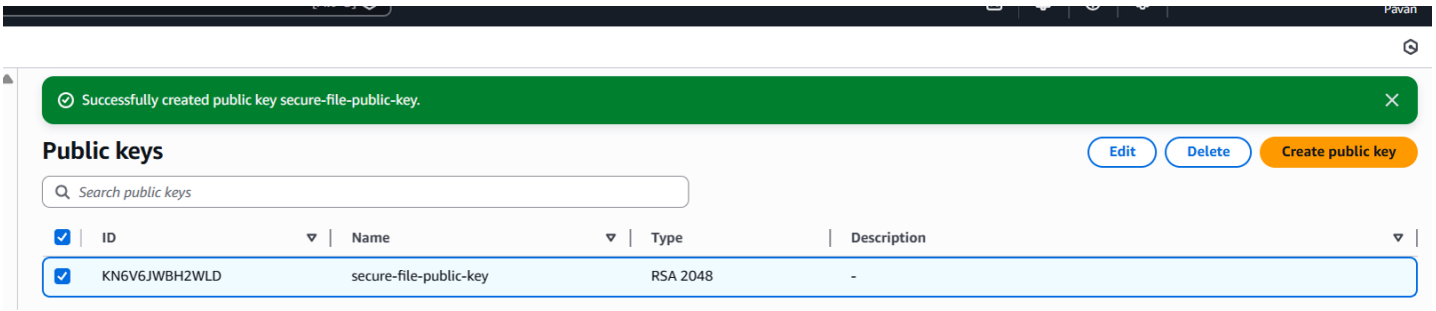
```
openssl genrsa -out private_key.pem 2048
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

✓ Result:

- **`private_key.pem` → keep SAFE (never upload)**
- **`public_key.pem` → upload to AWS**

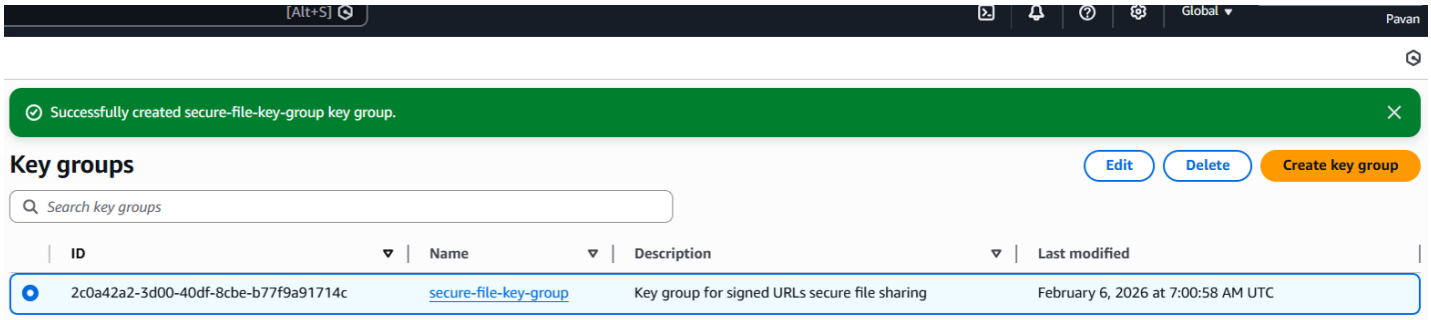
Phase 6 : UPLOAD PUBLIC KEY TO CLOUDFRONT

1. CloudFront → **Public keys**
2. Click **Create public key**
3. Name: **`secure-file-public-key`**
4. Paste contents of **`public_key.pem`**
5. Click **Create**



Phase 7: CREATE KEY GROUP

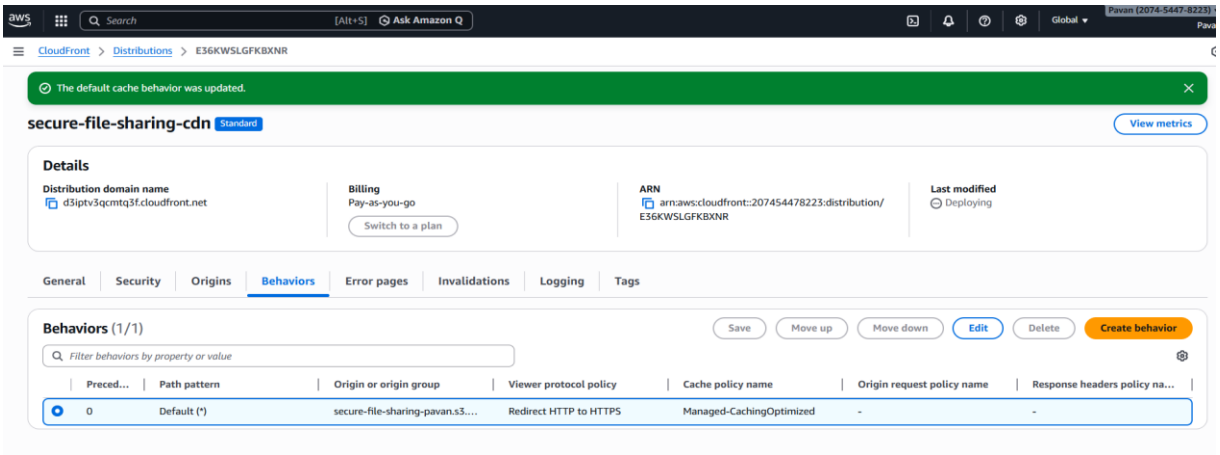
1. CloudFront → **Key groups**
2. Click **Create key group**
3. Name: **`secure-file-key-group`**
4. Add your **public key**
5. Create key group



Phase 8: ATTACH KEY GROUP TO CLOUDFRONT DISTRIBUTION

1. CloudFront → **Distributions**
2. Click your distribution
3. Go to **Behaviors**
4. Select default behavior → **Edit**
5. Scroll to **Restrict viewer access**
6. Choose:
   - ✅ **Yes – Use signed URLs or signed cookies**
   - Trusted key groups → select `secure-file-key-group`
7. Save changes

➢ Wait for deployment



Phase 9 : CREATE SIGNED URL

1. Open **CloudShell** and run:
   - **`nano generate_signed_url.py`**
2. CREATE PYTHON SCRIPT (paste the below code)

```
import datetime
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import padding
import base64

# ===== CONFIG =====
CLOUDFRONT_DOMAIN = "dxxxxxxxxxxxx.cloudfront.net"
OBJECT_PATH = "/private-files/sample.pdf"
KEY_PAIR_ID = "KN6V6JWBH2WLD"
PRIVATE_KEY_PATH = "private_key.pem"

# Expiry time (5 minutes)
expire_time = int((datetime.datetime.utcnow() + datetime.timedelta(minutes=5)).timestamp())

policy = f"""
{{
  "Statement": [
    {{
      "Resource": "https://{CLOUDFRONT_DOMAIN}{OBJECT_PATH}",
      "Condition": {{
        "DateLessThan": {{"AWS:EpochTime": {expire_time}}}
      }}
    }}
  ]
}}
"""

with open(PRIVATE_KEY_PATH, "rb") as key_file:
    private_key = serialization.load_pem_privat key(
        key_file.read(),
        password=None
    )

signature = private_key.sign(
    policy.encode(),
    padding.PKCS1v15(),
    hashes.SHA1()
)

encoded_signature = base64.b64encode(signature).decode()
encoded_policy = base64.b64encode(policy.encode()).decode()

signed_url = (
    f"https://{CLOUDFRONT_DOMAIN}{OBJECT_PATH}"
    f"?Policy={encoded_policy}"
    f"&Signature={encoded_signature}"
    f"&Key-Pair-Id={KEY_PAIR_ID}"
)

print("Signed URL:\n")
print(signed_url)
```

Replace in above code

- CLOUDFRONT_DOMAIN = "d3iptv3qcmtq3f.cloudfront.net"
- S3 OBJECT_PATH = "/Pavan_Resume__ECE.pdf"
- KEY_PAIR_ID = "KN6V6JWBH2WLD"

Save:

- **CTRL + O → Enter**
- **CTRL + X**

3. INSTALL REQUIRED LIBRARY
   - **`Run this → pip install cryptography`**
4. RUN SCRIPT
   - **`Run this → python3 generate_signed_url.py`**

5. You'll get a **long URL** like:
   Eg:(https://dxxxxx.cloudfront.net/private-files/sample.pdf?Policy=...&Signature=...&Key-Pair-Id=...)

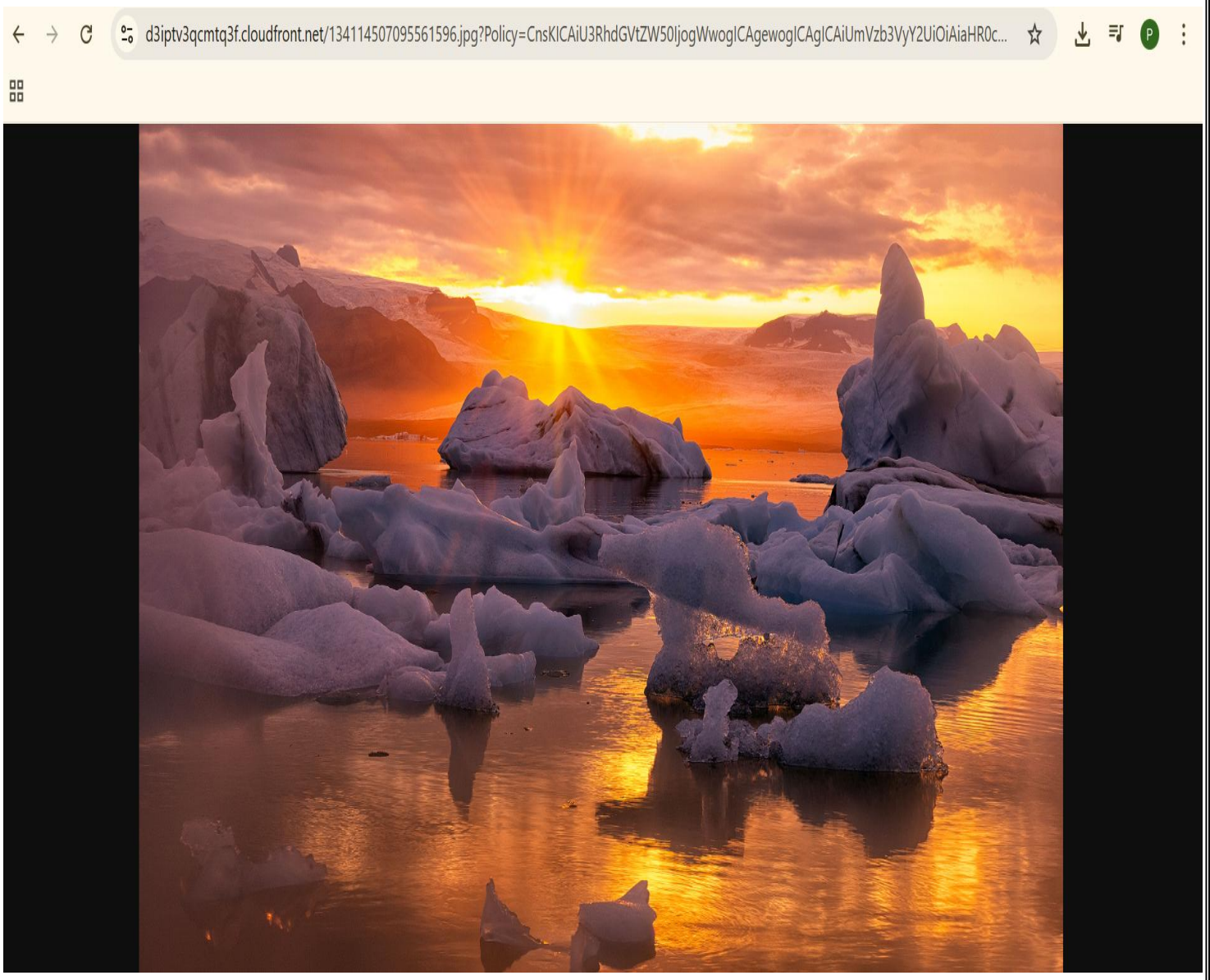
6. Copy the URL

```
----END PUBLIC KEY-----
$ nano generate_signed_url.py
$ pip install cryptography
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: cryptography in /usr/local/lib64/python3.9/site-packages (46.0.3)
Requirement already satisfied: typing-extensions>=4.13.2 in /usr/local/lib/python3.9/site-packages (from cryptography) (4.15.0)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib64/python3.9/site-packages (from cryptography) (2.0.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.9/site-packages (from cffi>=2.0.0->cryptography) (2.23)
$ python3 generate_signed_url.py
Signed URL:

https://d3iptv3qcmtq3f.cloudfront.net/Pavan_Resume__ECE.pdf?Policy=CnsKICAiU3RhdGVtZW50IjogWwogICAgewogICAgICAiUmVzb3VyY2UiOiAiaHR0cHM6Ly9kM2lwdHYzcWNtdHEzZi5jbC91ZGZyb250Lm5ldC9QYXZhbl9SZXN1bWVfX0VCRSSwZGYiLAogICAgICAiQ29uZGl0aW9uIjogewogICAgICAgICJEYXRlTGVzc1RoYW4iOiB7IkFXUzpFcG9jaFRpbWUiOiAxNzcwMzYyNjExfQogICAgICB9CiAgICBdCn0=&Signature=dqApq6gA5a74aWKfthEH+NHnJ4KLr15aWBwebzP6WNC1xhKMDxjBjkQX8ZHsz6YQZbRCq29t8O99NJABHUjelq0EMIB0b0q5DNjOh6gL61dqCumhGAU7XPkfSxmhpt7UKNlJQKDwkJN28TCuuim1k1alu1vZ8hkO3LjYw+82+qA+JKCVgJNI48OfNhZCLuHnCcxD8lXnShClMjnsBmqm3hRDYjdm1UgVfnc7XOLNklo9prwVWnQ
l6YDQZwMhlGZ6xgv/Oj82/O1sNtxymxujy+hvg8YEoX3AzHUUDwwnVefE9Z4VgKjPpLLOMFBon7Oohynq0m303TMjZ6/VgE0A==&Key-Pair-Id=KN6V6JWBH2WLD
$
```

Phase 10 : TEST THE SIGNED URL

1. **Open a new Incognito / Private window**
2. **Paste the copied signed URL**
3. Press **Enter**
    - File **downloads / opens successfully**



Troubleshooting:

Issue 1: Object not found error

Cause:

Incorrect object path used in signed URL (case-sensitive).

Solution:

Verify the exact object key name from S3 and ensure the same path is used in the signed URL.

Issue 2 : CloudFront returns Access Denied after enabling signed URLs

Cause:

Direct CloudFront URL access is blocked once signed URLs are enabled.

Solution:

Always access files using signed URLs only, not the direct CloudFront object URL.

Issue 3: Signed URL not working (403 Forbidden)

Cause:

Key Group is not attached to the CloudFront behavior or signed URLs are not enabled.

Solution:

Edit CloudFront behavior → Set Restrict viewer access = Yes → Select Trusted key groups → Attach the correct key group → Save changes.

Conclusion

The **Secure File Sharing using Amazon S3 and CloudFront** project successfully demonstrates how to deliver private files over the internet in a secure and controlled manner. By keeping the S3 bucket completely private and using CloudFront with **signed URLs**, access to files is restricted only to authorized users for a limited time. This approach ensures that sensitive documents are protected from unauthorized access while still providing fast and reliable content delivery through CloudFront's global CDN.

The project highlights the effective use of **IAM policies, Origin Access Control (OAC), key groups, and cryptographic key pairs** to enforce strong security at multiple layers. Overall, this solution is scalable, cost-effective, and suitable for real-world use cases such as secure document sharing, resume distribution, internal reports, and paid content delivery, making it a robust example of secure cloud architecture on AWS.